

Le but de cette séquence est de créer une interface graphique pour créer des applications plus interactives. Pour cela, nous allons utiliser une bibliothèque qui se nomme Tkinter dans un premier temps puis pygame dans une autre séquence.

1. Première application :

Le but est de créer une application qui affiche la fenêtre ci-contre. Lorsqu'on clique sur le bouton, cela ouvre la page web NSI. Dans un premier temps, commençons par importer la bibliothèque tkinter à l'aide de la commande :

```
from tkinter import *
```

Ensuite il faut créer la fenêtre que l'on va appeler window :

```
window= Tk()
```

La variable est créée mais il faut la personnaliser, nous souhaitons une fenêtre de 800x800 pixels bleu (couleur : #0050FF) dont le nom est « Mon application » la taille minimum de la fenêtre est 400x300 pixels.

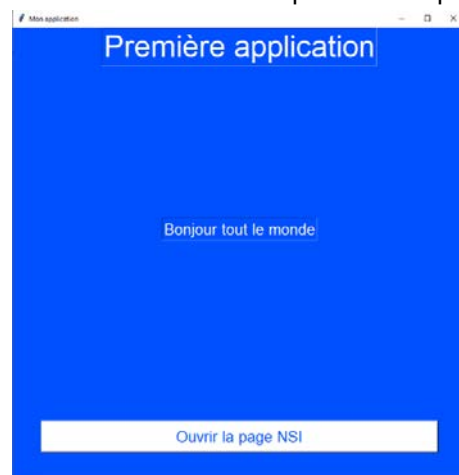
Compléter le code ci-dessous pour répondre à ce cahier des charges :

```
window.title("_____") #titre de la fenêtre
window.geometry("_____x_____") #dimension de la fenêtre
window.minsize(_____,_____) #taille minimum de la fenêtre dès qu'on l'a réduit
window.config(background='_____') #couleur du fond en RGB en hexadécimal
```

Pour afficher la fenêtre il faut utiliser la méthode mainloop() (cette ligne sera toujours à la fin du script)

A la fin de votre script rajouter **window.mainloop()**

Enregistrer sous Application1.py puis tester.



Maintenant, il faut créer des zones pour les textes que l'on appelle Frame. Les paramètres sont très variés, nous allons utiliser en premier le nom de la fenêtre où sont ces frames ici **window**, la couleur de fond **bg='#0050FF'**, l'épaisseur de la bordure en pixel **bd=1**, la bordure en relief pour la voir **relief=SUNKEN**.

Une fois que la zone est créée, il faut le positionner et l'afficher pour cela c'est la méthode **.pack()** qui peut avoir comme paramètres : **side=LEFT** ou droite : **RIGHT** ou haut : **TOP** ou bas : **BOTTOM** ou bien **expand=YES** pour la placer centre.

On peut aussi la positionner avec **padx** et **pady** (espace horizontal à gauche et droite pour pad x et haut et bas pour pady)

Compléter le code ci-dessous :

```
frame=Frame(window,bg='#0050FF', bd=1, relief=SUNKEN) #bordure bd=1 pixel et relief
frame2=Frame(_____,bg='_____', bd=_____, relief=_____) #bordure bd=1 pixel et relief
frame.pack(side=_____) #On positionne le titre en haut
frame2.pack(expand=_____) #on le place au centre
```

On va créer un texte **label_titre** qui a comme texte **Première application**, la font sera **courrier 40** pixels et la couleur arrière-plan **0050FF** et texte en blanc **FFFFFF**. Pour cela codez :

```
label_titre=Label(frame, text="Première application",font=("Courier",40),bg='#0050FF',fg='FFFFFF')
label_titre.pack(side=TOP) #affiche le texte ajouté et positionne le composant en haut
```

Compléter le sous-titre afin qui a comme texte **Bonjour tout le monde**, la font sera **courrier 20** pixels et la couleur arrière-plan (bg) **0050FF** et texte (fg) en blanc **FFFFFF**

```
label_sous_titre=Label(_____,text="_____",font=(_____,_____),bg='_____',fg='_____')
label_sous_titre.pack(side=_____) # affiche le texte ajouté et positionne le composant en haut
```

Enregistrer sous Application1.py puis tester.

Reste à ajouter le bouton, pour cela, il faudra la bibliothèque webbrowser en codant en début de script:

```
import webbrowser
```

puis une fonction qui ouvrira la page :

```
def ouvrir_nsi():
    webbrowser.open_new("http://gisele.bareux.free.fr/NSI1.htm ")
```

Pour créer le bouton, on retrouve la même structure que Label mais on mettra à la place Button. Il faudra rajouter un paramètre command qui dira au programme ce qu'il doit faire lorsqu'on clique sur le bouton.

Créer le bouton à l'aide des lignes de codes ci-dessous :

```
NSI_bouton=Button(window,text="Ouvrir la page NSI",font=("Courier",20),bg='white',fg='#0050FF',
command=ouvrir_nsi) #création du bouton et ouverture de la page avec command
```

```
NSI_bouton.pack(pady=50,fill=X, padx=50) # fill=X on remplit au maximum sur l'horizontal en laissant
50 pixels à cause de padx et 50 en-dessous de la fenêtre window à cause de pady
```

Enregistrer sous Application1.py puis tester.

Remarque : Si vous souhaitez créer une zone de texte où l'utilisateur tapera quelque chose, on mettra Entry.
Voilà la première application est terminée.

2. Création d'un générateur de mots de passe

Le but est de créer une application qui génère un mot de passe ci-contre.

Lorsqu'on clique sur le bouton ou que l'on clique sur Fichier puis Nouveau, cela génère un mot de passe.

Dans un premier temps, commençons par importer toutes les bibliothèques nécessaires :

```
tkinter
string
random
```



Créer une fenêtre windows qui a pour titre « Générateur de mot de passe » qui a pour dimensions 720x480 et comme couleur de fond #406599.

Dans cette fenêtre créer un frame avec le même fond. Pour cela compléter le code ci-dessous :

```
from _____ import *
from _____ import * #pour définir les caractères autorisés du mot de passe
from _____ import *
```

#Création de la fenêtre

```
window= Tk()
window.title("_____") #titre de la fenêtre
window.geometry("____x_____") #dimensions de la fenêtre
window.config(background='_____') #couleur du fond en RGB en hexadécimal
```

#création de la fenêtre principale

```
frame=Frame(_____,bg='_____')
```

Enregistrer sous Generateur_mdp.py puis tester.

Nous allons commencer par l'image login.png qui fera 300x300. N'oubliez pas de mettre le fichier [login.png](#) dans le même répertoire que le programme.

Compléter le code ci-dessous :

#création d'image

```
largeur = _____
hauteur = _____
```

subsample permet de réduire l'image de N fois sa taille en X et Y et zoom d'augmenter l'image de M fois donc en appliquant les deux, les dimensions de l'image seront multipliées par M/N

On souhaite réduire l'image de moitié, compléter de code ci-dessous en conséquence :

```
image=PhotoImage(file="_____").zoom(____).subsample(_____)
```

Création de l'espace graphique, pour cela on utilise un canvas dont on définit la fenêtre puis la largeur et la hauteur puis le background ainsi que bd=0 (bordure) et highlightthickness=0 (épaisseur de ligne)

```
canvas=Canvas(frame,width=_____,height=_____,bg='#_____',bd=0,highlightthickness=0)
canvas.create_image(_____,_____,image=image) #Coordonnées du centre de l'image
```

On utilise une grille au lieu de pack pour placer les éléments à gauche et droite de la fenêtre : row=ligne et column=colonne

L'image sera sur la ligne 0 et la colonne 0 alors que le reste sera dans un frame_droite qui sera à la ligne 0 mais colonne1.

```
canvas.grid(row=_____,column=_____)
```

Créer le frame_droite avec le même fond que précédemment

```
#Créer une sous boite à droite
frame_droite=Frame(frame,bg='#_____')
```

Dans ce frame_droite, créer le titre Mot de passe en blanc avec comme police Courier 20

```
#Création du titre
titre=Label(_____,text="_____",font=("_____",_____),bg='#_____',fg='_____')
titre.pack()
```

Dans ce frame_droite, créer le champ pour le mot de passe (à la place de label on met Entry) avec les mêmes paramètres que le titre.

```
#Création du champ pour la saisi du mot de passe
champ_mdp=Entry(_____,font=("_____",_____),bg='#_____',fg='_____')
champ_mdp.pack()
Enregistrer sous Generateur_mdp.py puis tester.
```

Dans ce frame_droite, créer le bouton « Générer le mot de passe' (à la place de label on met Button) avec les mêmes paramètres que le titre. On appellera la fonction générer_mdp lorsqu'on clique dessus. Le bouton remplira tout le frame en X et aura un pady de 20.

```
#Création du bouton
BP_mdp=Button(_____,text="_____",font=("_____",_____),bg='#_____',fg='_____',command=generer_mdp)
BP_mdp.pack(fill=_____,pady=_____)
```

Placer le frame_droite à la ligne0 et colonne 1

```
#On place la sous boite à droite de l'image
frame_droite.grid(row=_____,column=_____)
```

Afficher le frame centré

```
#afficher le frame
frame.pack(expand=YES)
```

Pour le moment, le programme génère une erreur car la fonction generer_mdp n'est pas créée. Nous allons créer une fonction qui génère un mot de passe de 6 à 12 caractères dont les caractères autorisés sont tout le code ascii + ponctuation + nombre pour cela il faut la bibliothèque string dans laquelle on va prendre les méthodes ascii_letters et punctuation et digits.

Le mdp va être créé grâce à une compréhension vu dans le chapitre S6.

Compléter le code ci-dessous :

```
def generer_mdp():
    mdp_mini=_____
    mdp_maxi=_____
    caracteres_autorises=_____+_____+_____ #on prend tout le code ascii + ponctuation +
    nombre.
    mdp=""
    for x in range(randint(_____,_____)):
        mdp+=choice(_____)
    champ_mdp.delete(0,END)#on efface le mot de passe s'il y a quelque chose dedans
    champ_mdp.insert(0,mdp)#on insert le mot de passe à l'indice 0
Enregistrer sous Generateur_mdp.py puis tester.
```

Reste le menu à créer. Il va être créé dans la fenêtre window et aura un tearoff=0 afin qu'il ne soit pas détachable de la fenêtre. Ensuite il suffit de rajouter les différents sous-menus avec la méthode add_command et add_cascade pour les sous-menus.

```
#Création d'une barre de menu
menu_barre=Menu(window)
```

#Créer un premier menu

```
menu_fichier=Menu(menu_barre,tearoff=_____) #menu fichier non détachable tearoff=0
menu_fichier.add_command(label="Nouveau",command=generer_mdp) #élément du menu fichier
menu_fichier.add_command(label="Quitter",command=window.destroy) #élément du menu fichier
menu_barre.add_cascade(label="Fichier",menu=menu_fichier) #élément de la barre de menu
```

#configurer notre fenêtre pour ajouter cette barre

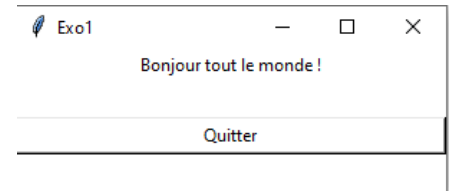
```
window.config(menu=menu_barre)
```

Enregistrer sous `Generateur_mdp.py` puis tester.

3. Exercices

Créer un programme qui affiche la fenêtre ci-contre. Le nom de la fenêtre est `Exo1`, elle a pour dimensions 300x100, tous les fond sont blanc et les écritures en noir. Lorsqu'on clique sur quitter, cela ferme la fenêtre.

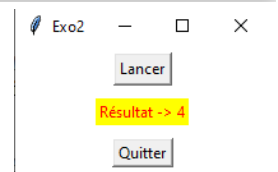
Coder ce cahier des charges, sauvegarder sous `Exo1.py` puis tester.



Créer un programme qui simule un lancé de dé. Le résultat du lancé sera dans un champ jaune et sera affiché à l'aide d'une fonction :
NouvelleLance : 'Resultat'+str(nb_tiré).

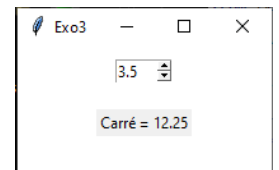
La fenêtre fera 200x100. Les deux boutons et le champ auront un `padx=5` et `pady=5`.

Coder ce cahier des charges, sauvegarder sous `Exo2.py` puis tester.



On peut aussi créer des champs de nombre avec incrément, c'est ce que l'on appelle `Spinbox` avec comme paramètres la fenêtre d'affichage, le mini, le maxi, l'incrément, la valeur du texte, la largeur et la commande effectuée à chaque clic. Pour récupérer la valeur du champ, il suffit de coder `valeur.get()`.

Exemple :



`Spinbox(Mafenetre,from_=0,to=10,increment=0.5,textvariable=Valeur,width=5,command=carre)`

Créer un programme qui calcule le carré d'un nombre défini par l'utilisateur.

La fenêtre fera 200x100. Le résultat et le champ auront un `padx=30` et `pady=10`.

Le résultat sera calculé par une fonction `carre()`

Coder ce cahier des charges, sauvegarder sous `Exo3.py` puis tester.

On peut aussi créer des curseurs à déplacer, c'est ce que l'on appelle `Scale` avec comme paramètres la fenêtre d'affichage, le mini, le maxi, la résolution, l'orientation (horizontal ou vertical), la longueur, la largeur, l'intervalle, l'étiquette et la valeur. On pourra aussi ajouter une commande si nécessaire.

Exemple :

`Echelle = Scale(Mafenetre,from_=-100,to=100,resolution=10,orient=HORIZONTAL,length=300,width=20,tickinterval=20, label="Curseur",variable=Valeur)`

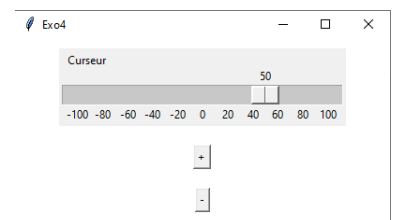
Créer un programme qui affiche la fenêtre ci-contre.

La fenêtre fera 400x200. Le résultat et le champ auront un `padx=10` et `pady=10`.

Les boutons + et - seront gérés par deux fonctions `plus()` et `moins()` qui ajouterons (ou soustrairons) 10 à la valeur courante du curseur.

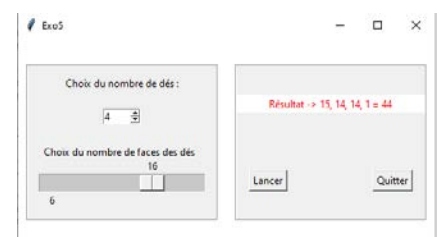
Pour avoir la valeur du curseur on utilise `Valeur.get()` et pour la modifier, on utilise `Valeur.set()`.

Coder ce cahier des charges, sauvegarder sous `Exo4.py` puis tester.



Réaliser une interface permettant de faire un lancé de plusieurs dés et d'afficher le résultat :

- Le nombre de dés est paramétré par un widget `entry` et doit être compris entre 1 et 6



- Le nombre de faces des dés imposé par un widget scale doit varier entre 6 et 20
- Les résultats sont affichés dans un label avec une virgule comme séparateur, vous afficherez également le total du lancé.
- La fenêtre fera 500x250.

Le bouton lancer exécutera la fonction NouveauLance() dont le code est le suivant :

Commenter la fonction ci-dessous :

```
def NouveauLance():
    NbDes=int(Valeur.get())
    resultat=""
    total =0
    max = int(NbFacesDes.get())
    for i in range (0,NbDes):
        nb = randint(1,max)
        total=total+nb
        resultat=resultat + str(nb)
        if i<NbDes-1:
            resultat=resultat+', '
    Texte.set('Résultat -> ' + resultat+' = '+str(total))
```

Coder ce cahier des charges, sauvegarder sous Exo5.py puis tester.

On peut aussi afficher un cercle à l'aide d'un Canvas.create_oval qui a comme paramètres les coordonnées x et y du point en haut à gauche puis celles du point en bas à gauche, le contour et le remplissage.

Exemple :

Canvas.create_oval(x-r, y-r, x+r, y+r, outline='blue', fill='blue') où x et y sont les coordonnées du centre et r le rayon.

Remarque : pour afficher un carré, on utiliserait Canvas.create_rectangle

Pour tout effacer, il suffit de coder **Canvas.delete(ALL)**

Créer le programme qui affiche la fenêtre ci-contre 500x400 dont le Canvas a pour dimensions 480x320,

- lorsqu'on clique sur Go, on affiche un cercle dans une position aléatoire à l'aide de la fonction Cercle(),
- lorsqu'on clique sur Effacer, on efface le canvas à l'aide de la fonction Effacer()
- lorsqu'on clique sur Quitter, on détruit le fenêtre.



Coder ce cahier des charges, sauvegarder sous Exo6.py puis tester.

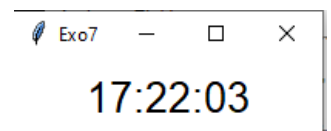
Nous allons continuer avec des animations, pour cela, il faudra utiliser la bibliothèque time. Dans cette bibliothèque, il y a une méthode after() qui appelle une fonction après une durée donnée en millisecondes.

Pour obtenir l'heure en heure, minutes et secondes, on utilise la méthode strftime('%H:%M:%S').

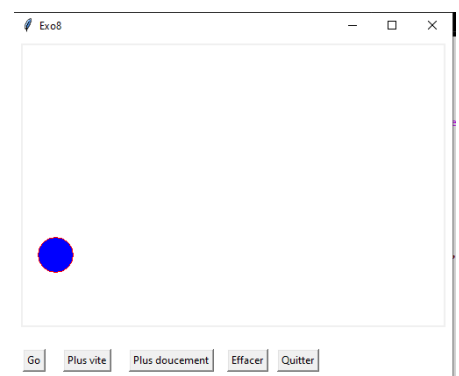
Pour modifier l'heure affichée on utilisera la fonction suivante :

```
def MAJHeure():
    # on arrive ici toutes les 1000 ms
    heure.set(strftime('%H:%M:%S'))
    window.after(1000,MAJHeure)
```

Créer le programme qui affiche la fenêtre ci-contre 200x50 puis sauvegarder sous Exo7.py puis tester.



- Reprendre l'exo6, modifier la fonction Cercle() pour déplacer le cercle après chaque valeur de la variable tempo.
- Lorsqu'il arrive sur une paroi :
 - Si x<r ou x>largeur-r alors déplacementX=-déplacementX,
 - si y<r ou y>hauteur-r alors déplacementY=-déplacementY
 - les déplacementX et Y seront initialisés aléatoirement avec un entier entre 1 et 5
 - La position X,Y du cercle au départ sera largeur/2 et hauteur/2 (au milieu du canvas)
- Créer une fonction PlusVite() qui diminue tempo de 5 jusqu'à à minimum de 5.
- Créer une fonction PlusDoucement() qui augmente tempo de 5.



✚ Créer deux boutons Plus vite et Plus doucement qui appellent les deux fonctions précédentes.

Créer le programme qui affiche la fenêtre ci-contre puis sauvegarder sous Exo8.py puis tester.